This is CS50

CS50's Introduction to Computer Science

OpenCourseWare

Donate (https://cs50.harvard.edu/donate)

David J. Malan (https://cs.harvard.edu/malan/)
malan@harvard.edu
(https://www.facebook.com/dmalan) (https://github.com/dmalan) (https://www.instagram.com/davidjmalan/) (https://www.linkedin.com/in/malan/) (https://www.reddit.com/user/davidjmalan) (https://x.com/davidjmalan)

Lecture 9

- Welcome!
- http-server
- Flask
- Forms
- Templates
- Request Methods
- Frosh IMs
- Flask and SQL
- Cookies and Session
- Shopping Cart
- Shows
- APIs
- JSON
- Summing Up

Welcome!

- In previous weeks, you have learned numerous programming languages, techniques, and strategies.
- Indeed, this class has been far less of a *C class* or *Python class* and far more of a *programming class*, such that you can go on to follow future trends.
- In these past several weeks, you have learned *how to learn* about programming.

- Today, we will be moving from HTML and CSS into combining HTML, CSS, SQL, Python, and JavaScript so you can create your own web applications.
- You might consider using the skills you learn this week to create your final project.

http-server

- Up until this point, all HTML you saw was pre-written and static.
- In the past, when you visited a page, the browser downloaded an HTML page, and you were able to view it. These are considered *static* pages, in that what is programmed in the HTML is exactly what the user sees and downloads *client-side* to their internet browser.
- Dynamic pages refer to the ability of Python and similar languages to create HTML on-the-fly. Accordingly, you can have web pages that are generated *server-side* by code based upon the input or behavior of users.
- You have used `http-server` in the past to serve your web pages. Today, we are going to utilize a new server that can parse out a web address and perform actions based on the URL provided.
- Further, last week, you saw URLs as follows:

  ```
  https://www.example.com/folder/file.html
  ```

  Notice that `file.html` is an HTML file inside a folder called `folder` at `example.com`.

Flask

- This week, we introduce the ability to engage with *routes* such as `https://www.example.com/route?key=value`, where specific functionality can be generated on the server via the keys and values provided in the URL.
- *Flask* is a third-party library that allows you to host web applications using the Flask framework, or a micro-framework, within Python.
- You can run Flask by executing `flask run` in your terminal window in cs50.dev (https://cs50.dev).
- To do so, you will need a file called `app.py` and another called `requirements.txt`. `app.py` contains code the tells Flask how to run your web application. `requirements.txt` includes a list of the libraries that are required for your Flask application to run.
- Here is a sample of `requirements.txt`:

  ```
  Flask
  ```

  Notice only `Flask` appears in this file. This is because Flask is required to run the Flask application.

- Here is a very simple Flask application in `app.py`:

```
# Says hello to world by returning a string of text

from flask import Flask

app = Flask(__name__)


@app.route("/")
def index():
    return "hello, world"
```

  Notice that the `/` route simply returns the text `hello, world`.

- We can also create code that implements HTML:

```
# Says hello to world by returning a string of HTML

from flask import Flask

app = Flask(__name__)


@app.route("/")
def index():
    return '<!DOCTYPE html><html lang="en"><head><title>hello</title></head><b
```

  Notice that rather than returning simple text, this provides HTML.

- Improving our application, we can also serve HTML based upon templates by creating a folder called `templates` and creating a file called `index.html` with the following code within that folder:

```
<!DOCTYPE html>

<html lang="en">

    <head>
        <title>hello</title>
    </head>

    <body>
        hello, world
    </body>

</html>
```

- Then, in the same folder that the `templates` folder appears, create a file called `app.py` and add the following code:

```
# Uses request.args.get

from flask import Flask, render_template, request
```

```
app = Flask(__name__)


@app.route("/")
def index():
    name = request.args.get("name", "world")
    return render_template("index.html", name=name)
```

Notice that this code defines `app` as the Flask application. Then, it defines the `/` route of `app` as returning the contents of `index.html` with the argument of `name`. By default, the `request.args.get` function will look for the `name` being provided by the user. If no name is provided, it will default to `world`. `@app.route` is otherwise known as a decorator.

- You can run this web application by typing `flask run` in the terminal window. If Flask does not run, ensure that your syntax is correct in each of the files above. Further, if Flask will not run, make sure your files are organized as follows:

```
/templates
    index.html
app.py
requirements.txt
```

- Once you get it running, you will be prompted to click a link. Once you navigate to that webpage, try adding `?name=[Your Name]` to the base URL in your browser's URL bar.

Forms

- Improving upon our program, we know that most users will not type arguments into the address bar. Instead, programmers rely upon users to fill out forms on web pages. Accordingly, we can modify `index.html` as follows:

```
<!DOCTYPE html>

<html lang="en">

    <head>
        <title>hello</title>
    </head>

    <body>
        <form action="/greet" method="get">
            <input autocomplete="off" autofocus name="name" placeholder="Name"
            <button type="submit">Greet</button>
        </form>
    </body>

</html>
```

Notice that a form is now created that takes the user's name and then passes it off to a route called `/greet`. `autocomplete` is turned off. Further, a `placeholder` with the text `name` is included.

- Further, we can change `app.py` as follows:

```
# Adds a form, second route

from flask import Flask, render_template, request

app = Flask(__name__)


@app.route("/")
def index():
    return render_template("index.html")


@app.route("/greet")
def greet():
    return render_template("greet.html", name=request.args.get("name", "world"
```

  Notice that the default path will display a form for the user to input their name. The `/greet` route will pass the `name` to that web page.

- To finalize this implementation, you will need another template for `greet.html` in the `templates` folder as follows:

```
<!DOCTYPE html>

<html lang="en">

    <head>
        <meta name="viewport" content="initial-scale=1, width=device-width">
        <title>hello</title>
    </head>

    <body>
        hello, {{ name }}
    </body>

</html>
```

  Notice that this route will now render the greeting to the user, followed by their name.

Templates

- Both of our web pages, `index.html` and `greet.html`, have much of the same data. Wouldn't it be nice to allow the body to be unique but copy the same layout from page to page?
- First, create a new template called `layout.html` and write code as follows:

```
<!DOCTYPE html>

<html lang="en">

    <head>
```

```
        <title>hello</title>
    </head>

    <body>
        {% block body %}{% endblock %}
    </body>

</html>
```

Notice that the `{% block body %}{% endblock %}` allows for the insertion of other code from other HTML files.

- Then, modify your `index.html` as follows:

```
{% extends "layout.html" %}

{% block body %}

    <form action="/greet" method="get">
        <input autocomplete="off" autofocus name="name" placeholder="Name" typ
        <button type="submit">Greet</button>
    </form>

{% endblock %}
```

Notice that the line `{% extends "layout.html" %}` tells the server where to get the layout of this page. Then, the `{% block body %}{% endblock %}` tells what code to be inserted into `layout.html`.

- Finally, change `greet.html` as follows:

```
{% extends "layout.html" %}

{% block body %}
    hello, {{ name }}
{% endblock %}
```

Notice how this code is shorter and more compact.

Request Methods

- You can imagine scenarios where it is not safe to utilize `get`, as usernames and passwords would show up in the URL.
- We can utilize the method `post` to help with this problem by modifying `app.py` as follows:

```
# Switches to POST

from flask import Flask, render_template, request

app = Flask(__name__)
```

```
@app.route("/")
def index():
    return render_template("index.html")


@app.route("/greet", methods=["POST"])
def greet():
    return render_template("greet.html", name=request.form.get("name", "world"
```

Notice that `POST` is added to the `/greet` route, and that we use `request.form.get` rather than `request.args.get`.

- This tells the server to look *deeper* into the virtual envelope and not reveal the items in `post` in the URL.
- Still, this code can be advanced further by utilizing a single route for both `get` and `post`. To do this, modify `app.py` as follows:

```
# Uses a single route

from flask import Flask, render_template, request

app = Flask(__name__)


@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        return render_template("greet.html", name=request.form.get("name", "wo
    return render_template("index.html")
```

Notice that both `get` and `post` are done in a single routing. However, `request.method` is utilized to properly route based on the type of routing requested by the user.

- Accordingly, you can modify your `index.html` as follows:

```
{% extends "layout.html" %}

{% block body %}

    <form action="/" method="post">
        <input autocomplete="off" autofocus name="name" placeholder="Name" typ
        <button type="submit">Greet</button>
    </form>

{% endblock %}
```

Notice that the form `action` is changed.

- Still, there is a bug in this code. With our new implementation, when someone types in no name into the form, `Hello,` is displayed without a name. We can improve our code by editing `app.py` as follows:

```
# Moves default value to template

from flask import Flask, render_template, request

app = Flask(__name__)


@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        return render_template("greet.html", name=request.form.get("name"))
    return render_template("index.html")
```

Notice that `name=request.form.get("name"))` is changed.

- Finally, change `greet.html` as follows:

```
{% extends "layout.html" %}

{% block body %}

    hello,
    {% if name -%}
        {{ name }}
    {%- else -%}
        world
    {%- endif %}

{% endblock %}
```

Notice how `hello, {{ name }}` is changed to allow for a default output when no name is identified.

- As we've been changing many files, you may wish to compare your final code with our final code (https://cdn.cs50.net/2025/fall/lectures/9/src9/hello10/).

Frosh IMs

- Frosh IMs or *froshims* is a web application that allows students to register for intramural sports.
- Close all your `hello` related windows and create a folder by typing `mkdir froshims` in the terminal window. Then, type `cd froshims` to browse to this folder. Within, create a directory called templates by typing `mkdir templates`.
- Next, in the `froshims` folder, type `code requirements.txt` and code as follows:

```
Flask
```

As before, Flask is required to run a Flask application.

- Finally, type `code app.py` and write code as follows:

```
# Implements a registration form using a select menu, validating sport server-
```

```
from flask import Flask, render_template, request

app = Flask(__name__)

SPORTS = [
    "Basketball",
    "Soccer",
    "Ultimate Frisbee"
]


@app.route("/")
def index():
    return render_template("index.html", sports=SPORTS)


@app.route("/register", methods=["POST"])
def register():

    # Validate submission
    if not request.form.get("name") or request.form.get("sport") not in SPORTS
        return render_template("failure.html")

    # Confirm registration
    return render_template("success.html")
```

Notice that a `failure` option is provided, such that a failure message will be displayed to the user if the `name` or `sport` field is not properly filled out.

- Next, create a file in the `templates` folder called `index.html` by typing `code templates/index.html` and write code as follows:

```
{% extends "layout.html" %}

{% block body %}
    <h1>Register</h1>
    <form action="/register" method="post">
        <input autocomplete="off" autofocus name="name" placeholder="Name" typ
        <select name="sport">
            <option selected value="">Sport</option>
            {% for sport in sports %}
                <option value="{{ sport }}">{{ sport }}</option>
            {% endfor %}
        </select>
        <button type="submit">Register</button>
    </form>
{% endblock %}
```

- Next, create a file called `layout.html` by typing `code templates/layout.html` and write code as follows:

```
<!DOCTYPE html>

<html lang="en">
```

```
    <head>
        <meta name="viewport" content="initial-scale=1, width=device-width">
        <title>froshims</title>
    </head>

    <body>
        {% block body %}{% endblock %}
    </body>

</html>
```

- Fourth, create a file in templates called `success.html` as follows:

```
{% extends "layout.html" %}

{% block body %}
    You are registered!
{% endblock %}
```

- Finally, create a file in templates called `failure.html` as follows:

```
{% extends "layout.html" %}

{% block body %}
    You are not registered!
{% endblock %}
```

- Execute `flask run` and check out the application at this stage.
- You can imagine how we might want to see the various registration options using radio buttons. We can improve `index.html` as follows:

```
{% extends "layout.html" %}

{% block body %}
    <h1>Register</h1>
    <form action="/register" method="post">
        <input autocomplete="off" autofocus name="name" placeholder="Name" typ
        {% for sport in sports %}
            <input name="sport" type="radio" value="{{ sport }}"> {{ sport }}
        {% endfor %}
        <button type="submit">Register</button>
    </form>
{% endblock %}
```

  Notice how `type` has been changed to `radio`.
- Again, executing `flask run` you can see how the interface has now changed.
- You can imagine how we might want to accept the registration of many different registrants. We can improve `app.py` as follows:

```
# Implements a registration form, storing registrants in a dictionary, with er

from flask import Flask, redirect, render_template, request
```

```
app = Flask(__name__)

REGISTRANTS = {}

SPORTS = [
    "Basketball",
    "Soccer",
    "Ultimate Frisbee"
]


@app.route("/")
def index():
    return render_template("index.html", sports=SPORTS)


@app.route("/register", methods=["POST"])
def register():

    # Validate name
    name = request.form.get("name")
    if not name:
        return render_template("error.html", message="Missing name")

    # Validate sport
    sport = request.form.get("sport")
    if not sport:
        return render_template("error.html", message="Missing sport")
    if sport not in SPORTS:
        return render_template("error.html", message="Invalid sport")

    # Remember registrant
    REGISTRANTS[name] = sport

    # Confirm registration
    return redirect("/registrants")


@app.route("/registrants")
def registrants():
    return render_template("registrants.html", registrants=REGISTRANTS)
```

Notice that a dictionary called `REGISTRANTS` is used to log the `sport` selected by `REGISTRANTS[name]`. Also, notice that `registrants=REGISTRANTS` passes the dictionary on to this template.

- Additionally, we can implement `error.html`:

```
{% extends "layout.html" %}

{% block body %}
    <h1>Error</h1>
    <p>{{ message }}</p>
    <img alt="Grumpy Cat" src="/static/cat.jpg">
{% endblock %}
```

- Further, create a new template called `registrants.html` as follows:

```
{% extends "layout.html" %}

{% block body %}
    <h1>Registrants</h1>
    <table>
        <thead>
            <tr>
                <th>Name</th>
                <th>Sport</th>
            </tr>
        </thead>
        <tbody>
            {% for name in registrants %}
                <tr>
                    <td>{{ name }}</td>
                    <td>{{ registrants[name] }}</td>
                </tr>
            {% endfor %}
        </tbody>
    </table>
{% endblock %}
```

Notice that `{% for name in registrants %}...{% endfor %}` will iterate through each of the registrants. Very powerful to be able to iterate on a dynamic web page!

- Finally, create a folder called `static` in the same folder as `app.py`. There, upload the following file of a cat (https://cdn.cs50.net/2025/fall/lectures/9/src9/froshims4/static/cat.jpg).
- Execute `flask run` and play with the application.
- You now have a web application! However, there are some security flaws! Because everything is held in memory, an adversary could change the HTML and *hack* the website. Further, this data will not persist if the server is shut down. Could there be some way we could have our data persist even when the server restarts?

Flask and SQL

- Just as we have seen how Python can interface with a SQL database, we can combine the power of Flask, Python, and SQL to create a web application where data will persist!
- To implement this, you will need to take a number of steps.
- First, download the following SQL database (https://cdn.cs50.net/2025/fall/lectures/9/src9/froshims7/froshims.db) into your `froshims` folder.
- Execute in the terminal `sqlite3 froshims.db` and type `.schema` to see the contents of the database file. Further type `SELECT * FROM registrants;` to learn about the contents. You'll notice that there are currently no registrations in the file.
- Next, modify `requirements.txt` as follows:

```
cs50
```

```
Flask
```

- Modify `index.html` as follows:

```
{% extends "layout.html" %}

{% block body %}
    <h1>Register</h1>
    <form action="/register" method="post">
        <input autocomplete="off" autofocus name="name" placeholder="Name" typ
        {% for sport in sports %}
            <input name="sport" type="checkbox" value="{{ sport }}"> {{ sport
        {% endfor %}
        <button type="submit">Register</button>
    </form>
{% endblock %}
```

- Modify `layout.html` as follows:

```
<!DOCTYPE html>

<html lang="en">

    <head>
        <meta name="viewport" content="initial-scale=1, width=device-width">
        <title>froshims</title>
    </head>

    <body>
        {% block body %}{% endblock %}
    </body>

</html>
```

- Ensure `error.html` appears as follows:

```
{% extends "layout.html" %}

{% block body %}
    <h1>Error</h1>
    <p>{{ message }}</p>
    <img alt="Grumpy Cat" src="/static/cat.jpg">
{% endblock %}
```

- Modify `registrants.html` to appear as follows:

```
{% extends "layout.html" %}

{% block body %}
    <h1>Registrants</h1>
    <table>
        <thead>
            <tr>
                <th>Name</th>
                <th>Sport</th>
                <th></th>
            </tr>
```

```
        </thead>
        <tbody>
            {% for registrant in registrants %}
                <tr>
                    <td>{{ registrant.name }}</td>
                    <td>{{ registrant.sport }}</td>
                    <td>
                        <form action="/deregister" method="post">
                            <input name="id" type="hidden" value="{{ registran
                            <button type="submit">Deregister</button>
                        </form>
                    </td>
                </tr>
            {% endfor %}
        </tbody>
    </table>
{% endblock %}
```

Notice that a hidden value `registrant.id` is included such that it's possible to use this `id` later in `app.py`

- Finally, modify `app.py` as follows:

```
# Implements a registration form, storing registrants in a SQLite database, wi

from cs50 import SQL
from flask import Flask, redirect, render_template, request

app = Flask(__name__)

db = SQL("sqlite:///froshims.db")

SPORTS = [
    "Basketball",
    "Soccer",
    "Ultimate Frisbee"
]


@app.route("/")
def index():
    return render_template("index.html", sports=SPORTS)


@app.route("/deregister", methods=["POST"])
def deregister():

    # Forget registrant
    id = request.form.get("id")
    if id:
        db.execute("DELETE FROM registrants WHERE id = ?", id)
    return redirect("/registrants")


@app.route("/register", methods=["POST"])
def register():

    # Validate name
```

```
    name = request.form.get("name")
    if not name:
        return render_template("error.html", message="Missing name")

    # Validate sports
    sports = request.form.getlist("sport")
    if not sports:
        return render_template("error.html", message="Missing sport")
    for sport in sports:
        if sport not in SPORTS:
            return render_template("error.html", message="Invalid sport")

    # Remember registrant
    for sport in sports:
        db.execute("INSERT INTO registrants (name, sport) VALUES(?, ?)", name,

    # Confirm registration
    return redirect("/registrants")


@app.route("/registrants")
def registrants():
    registrants = db.execute("SELECT * FROM registrants")
    return render_template("registrants.html", registrants=registrants)
```

Notice that the `cs50` library is utilized. A route is included for `register` for the `post` method. This route will take the name and sport taken from the registration form and execute a SQL query to add the `name` and the `sport` to the `registrants` table. The `deregister` routes to a SQL query that will grab the user's `id` and utilize that information to deregister this individual.

- You can execute `flask run` and examine the result.
- If you want to download our implementation of `froshims` you can do so here (https://cdn.cs50.net/2025/fall/lectures/9/src9/froshims7/).
- You can read more about Flask in the Flask documentation (https://flask.palletsprojects.com).

Cookies and Session

- `app.py` is considered a *controller*. A *view* is considered what the users see. A *model* is how data is stored and manipulated. Together, this is referred to as *MVC* (model, view, controller).
- While the prior implementation of `froshims` is useful from an administrative standpoint, where a back-office administrator could add and remove individuals from the database, one can imagine how this code is not safe to implement on a public server.
- For one, bad actors could make decisions on behalf of other users by hitting the deregister button – effectively deleting their recorded answer from the server.

- Web services like Google use login credentials to ensure users only have access to the right data.
- We can actually implement this itself using *cookies*. Cookies are small pieces of data that are stored in your browser such that your browser can communicate with the server and effectively say, "I'm an authorized user that has already logged in." This authorization through this cookie is called a *session*.
- Cookies may be stored as follows:

```
GET / HTTP/2
Host: accounts.google.com
Cookie: session=value
```

  Here, a `session` id is stored with a particular `value` representing that session.
- In the simplest form, we can implement this by creating a folder called `login` and then adding the following files.
- First, create a file called `requirements.txt` that reads as follows:

```
Flask
Flask-Session
```

  Notice that in addition to `Flask`, we also include `Flask-Session`, which is required to support login sessions.
- Second, in a `templates` folder, create a file called `layout.html` that appears as follows:

```
<!DOCTYPE html>

<html lang="en">

    <head>
        <meta name="viewport" content="initial-scale=1, width=device-width">
        <title>login</title>
    </head>

    <body>
        {% block body %}{% endblock %}
    </body>

</html>
```

  Notice this provides a very simple layout with a title and a body.
- Third, create a file in the `templates` folder called `index.html` that appears as follows:

```
{% extends "layout.html" %}

{% block body %}

    {% if name -%}
        You are logged in as {{ name }}. <a href="/logout">Log out</a>.
    {%- else -%}
        You are not logged in. <a href="/login">Log in</a>.
    {%- endif %}
```

```
{% endblock %}
```

Notice that this file looks to see if `session["name"]` exists (elaborated further in `app.py` below). If it does, it will display a welcome message. If not, it will recommend you browse to a page to log in.

- Fourth, create a file called `login.html` and add the following code:

```
{% extends "layout.html" %}

{% block body %}

    <form action="/login" method="post">
        <input autocomplete="off" autofocus name="name" placeholder="Name" typ
        <button type="submit">Log In</button>
    </form>

{% endblock %}
```

Notice this is the layout of a basic login page.

- Finally, create a file called `app.py` and write code as follows:

```
from flask import Flask, redirect, render_template, request, session
from flask_session import Session

# Configure app
app = Flask(__name__)

# Configure session
app.config["SESSION_PERMANENT"] = False
app.config["SESSION_TYPE"] = "filesystem"
Session(app)


@app.route("/")
def index():
    return render_template("index.html", name=session.get("name"))


@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        session["name"] = request.form.get("name")
        return redirect("/")
    return render_template("login.html")


@app.route("/logout")
def logout():
    session.clear()
    return redirect("/")
```

Notice the modified *imports* at the top of the file, including `session`, which will allow you to support sessions. Most importantly, notice how `session["name"]` is used in the `login`

and `logout` routes. The `login` route will assign the login name provided and assign it to `session["name"]`. However, in the `logout` route, the logging out is implemented by clearing the value of `session`.

- The `session` abstraction allows you to ensure only a specific user has access to specific data and features in our application. It allows you to ensure that no one acts on behalf of another user, for good or bad!
- If you wish, you can download our implementation (https://cdn.cs50.net/2025/fall/lectures/9/src9/login/) of `login`.
- You can read more about sessions in the Flask documentation (https://flask.palletsprojects.com/en/stable/api/#flask.session).

Shopping Cart

- Moving on to a final example of utilizing Flask's ability to enable a session.
- We examined the following code for `store` in `app.py`. The following code was shown:

```
from cs50 import SQL
from flask import Flask, redirect, render_template, request, session
from flask_session import Session

# Configure app
app = Flask(__name__)

# Connect to database
db = SQL("sqlite:///store.db")

# Configure session
app.config["SESSION_PERMANENT"] = False
app.config["SESSION_TYPE"] = "filesystem"
Session(app)


@app.route("/")
def index():
    books = db.execute("SELECT * FROM books")
    return render_template("books.html", books=books)


@app.route("/cart", methods=["GET", "POST"])
def cart():

    # Ensure cart exists
    if "cart" not in session:
        session["cart"] = []

    # POST
    if request.method == "POST":
        book_id = request.form.get("id")
        if book_id:
            session["cart"].append(book_id)
        return redirect("/cart")
```

```
    # GET
    books = db.execute("SELECT * FROM books WHERE id IN (?)", session["cart"])
    return render_template("cart.html", books=books)
```

Notice that `cart` is implemented using a list. Items can be added to this list using the `Add to Cart` buttons in `books.html`. When clicking such a button, the `post` method is invoked, where the `id` of the item is appended to the `cart`. When viewing the cart, invoking the `get` method, SQL is executed to display a list of the books in the cart.

- We also saw the contents of `books.html`:

```
{% extends "layout.html" %}

{% block body %}

    <h1>Books</h1>
    {% for book in books %}
        <h2>{{ book["title"] }}</h2>
        <form action="/cart" method="post">
            <input name="id" type="hidden" value="{{ book['id'] }}">
            <button type="submit">Add to Cart</button>
        </form>
    {% endfor %}

{% endblock %}
```

Notice how this creates the ability to `Add to Cart` for each book using `for book in books`.

- You can see the rest of the files that power this `flask` implementation in the source code (https://cdn.cs50.net/2025/fall/lectures/9/src9/store/).

Shows

- We looked at a pre-designed program called `shows`, in `app.py`:

```
# Searches for shows using LIKE

from cs50 import SQL
from flask import Flask, render_template, request

app = Flask(__name__)

db = SQL("sqlite:///shows.db")


@app.route("/")
def index():
    return render_template("index.html")


@app.route("/search")
def search():
```

```
    shows = db.execute("SELECT * FROM shows WHERE title LIKE ?", "%" + request
    return render_template("search.html", shows=shows)
```

Notice how the `search` route allows for a way by which to search for a `show`. This search looks for titles `LIKE` the one provided by the user.

- We also examined `index.html`:

```
{% extends "layout.html" %}

{% block body %}

    <form action="/search" method="get">
        <input autocomplete="off" autofocus name="q" placeholder="Query" type=
        <button type="submit">Search</button>
    </form>

{% endblock %}
```

- You can see the rest of the files of this implementation in the source code (https://cdn.cs50.net/2025/fall/lectures/9/src9/shows1/).

APIs

- An *application program interface* or *API* is a series of specifications that allow you to interface with another service. For example, we could utilize IMDB's API to interface with their database. We might even integrate APIs for handling specific types of data downloadable from a server.
- Improving upon `shows`, looking at an improvement of `app.py`, we saw the following:

```
# Searches for shows using Ajax

from cs50 import SQL
from flask import Flask, render_template, request

app = Flask(__name__)

db = SQL("sqlite:///shows.db")


@app.route("/")
def index():
    return render_template("index.html")


@app.route("/search")
def search():
    q = request.args.get("q")
    if q:
        shows = db.execute("SELECT * FROM shows WHERE title LIKE ? LIMIT 50",
```

```
    else:
        shows = []
    return render_template("search.html", shows=shows)
```

Notice that the `search` route executes a SQL query.

- Looking at `search.html`, you'll notice that it is very simple:

```
{% for show in shows %}
    <li>{{ show["title"] }}</li>
{% endfor %}
```

Notice that it provides a bulleted list.

- Finally, looking at `index.html`, notice that *AJAX* code is utilized to power the search:

```
<!DOCTYPE html>

<html lang="en">

    <head>
        <meta name="viewport" content="initial-scale=1, width=device-width">
        <title>shows</title>
    </head>

    <body>

        <input autocomplete="off" autofocus placeholder="Query" type="search">

        <ul></ul>

        <script>
            let input = document.querySelector('input');
            input.addEventListener('input', async function() {
                let response = await fetch('/search?q=' + input.value);
                let shows = await response.text();
                document.querySelector('ul').innerHTML = shows;
            });
        </script>

    </body>

</html>
```

Notice an event listener is utilized to dynamically query the server to provide a list that matches the title provided. This will locate the `ul` tag in the HTML and modify the web page accordingly to include the list of the matches.

- You can read more in the AJAX documentation (https://api.jquery.com/category/ajax/).

JSON

- *JavaScript Object Notation* or *JSON* is a text file of dictionaries with keys and values. This is a raw, computer-friendly way to get lots of data.
- JSON is a very useful way of getting back data from the server.
- You can see this in action in the `index.html` we examined together:

```
<!DOCTYPE html>

<html lang="en">

    <head>
        <meta name="viewport" content="initial-scale=1, width=device-width">
        <title>shows</title>
    </head>

    <body>

        <input autocomplete="off" autofocus placeholder="Query" type="text">

        <ul></ul>

        <script>
            let input = document.querySelector('input');
            input.addEventListener('input', async function() {
                let response = await fetch('/search?q=' + input.value);
                let shows = await response.json();
                let html = '';
                for (let i in shows) {
                    let title = shows[i].title.replace('<', '&lt;').replace('&
                    html += '<li>' + title + '</li>';
                }
                document.querySelector('ul').innerHTML = html;
            });
        </script>

    </body>

</html>
```

While the above may be somewhat cryptic, it provides a starting point for you to research JSON on your own to see how it can be implemented in your own web applications.

- Further, we examined `app.py` to see how the JSON response is obtained:

```
# Searches for shows using Ajax with JSON

from cs50 import SQL
from flask import Flask, jsonify, render_template, request

app = Flask(__name__)

db = SQL("sqlite:///shows.db")


@app.route("/")
def index():
```

```
    return render_template("index.html")


@app.route("/search")
def search():
    q = request.args.get("q")
    if q:
        shows = db.execute("SELECT * FROM shows WHERE title LIKE ? LIMIT 50",
    else:
        shows = []
    return jsonify(shows)
```

Notice how `jsonify` is used to convert the result into a readable format acceptable by contemporary web applications.

- You can read more in the JSON documentation (https://www.json.org/json-en.html).
- In summary, you now have the ability to complete your own web applications using Python, Flask, HTML, and SQL.

Summing Up

In this lesson, you learned how to utilize Python, SQL, and Flask to create web applications. Specifically, we discussed...

- Flask
- Forms
- Templates
- Request Methods
- Flask and SQL
- Cookies and Session
- APIs
- JSON